Assessment Task 3: Data Analytics in Action

Name: **Rafia Tasneem**

ID: **24869289**

Submitted to: Dr Maoying Qiao

Date:24/05/2024

Abstract:

The goal of this study was to create a classifier that could predict the "SpType-ELS" property, which denotes the estimated spectral class determined by Gaia and is classified as either "A" or "B." It used a large dataset of stellar attributes to test several classification methods, such as Logistic Regression, K-Neighbors, Random Forest, Decision Tree, Support Vector Classifier, and XGBoost Classifier. The best-performing classifier was found through extensive data preparation, hyperparameter adjustment, and model assessment. With its great accuracy, the final model proved to be a useful tool for data mining in astrophysical applications and offered insightful information on star classification. This research highlights the significance of machine learning to the advancement of astronomy.

I. Introduction:

In data science, it is essential to extract valuable insights from large datasets. This research aims to predict the spectral class of stars, classified as 'A' or 'B' based on the "SpType-ELS" characteristic, by using data mining algorithms. For the purpose of astrophysics study, precise spectrum categorization is important for understanding star attributes like temperature, brightness, and evolutionary stage.

The study included variety of classification techniques, including Support Vector Classifier, Random Forest, K-Neighbors, Logistic Regression, and Decision Tree, using an extensive dataset of stellar properties. The project involved careful model evaluation, hyperparameter optimization, and data preprocessing. In addition to achieving excellent classification accuracy, the goal of this research is to provide insightful information on the stellar classification process.

II. Dataset Overview:

The training dataset and the unknown dataset are the two halves of the dataset. Model training and validation are conducted using labeled data that has the "SpType-ELS" property applied, classifying stars as either 'A' or 'B'. In contrast, the unknown dataset has no labels and is meant to be used in conjunction with the trained model to generate predictions. In order to forecast the "SpType-ELS" characteristic, the dataset includes a number of attributes pertaining to stellar qualities. Physical properties like Teff, Rad, and Lum-Flame, as well as astrometric parameters like RA_ICRS and DE_ICRS and photometric data like Gmag, BPmag, and RPmag, are important aspects. The dataset also includes error measurements for several variables as well as derived features like BP-RP and BP-G. To create robust classifiers that properly predict the spectrum class, proper preprocessing such as handling missing values and normalizing data, is essential.

III. Data Mining Problem:

This main goal of this project was to create a classifier that could predict the "SpType-ELS" characteristic in each dataset. The classification task involved identifying the estimated spectral class (labeled as 'A' or 'B') that Gaia has detected. Since it aids in the study of numerous star features and behaviors, the precise prediction of this attribute is crucial for astrophysical research. Precise spectral classification can supplement more general astronomical investigations and improve the accuracy of stellar models by offering information about the temperature, brightness, and evolutionary stage of the star. The development of a trustworthy and reliable classifier for this purpose will promote scientific investigation and shows how data mining methods are used in real-world astronomical applications.

IV. Data Preprocessing and Transformations:

To guarantee data quality and relevance for model training, the original dataset underwent a number of preprocessing processes. These actions included encoding categorical variables, addressing missing values, and normalizing features. These preprocessing procedures are described in depth in the following sections.

1. Handling Missing Values:

When searching the dataset for missing values, it was discovered that column ‘pscol’ contained 143,685 missing entries, while columns ‘GRVSmag’ and ‘e_GRVSmag’ had 64,054 missing values each. Furthermore, there were differing amounts of missing values in columns ‘Lum-Flame’, ‘Mass-Flame’, ‘Age-Flame’, and ‘z-Flame’, ranging from 2,995 to 38,016. A systematic imputation approach was used to fill in these gaps. To ensure a robust estimation and reduce the impact of outliers, numerical columns were imputed using the median value of their respective distributions. To maintain the integrity of the data structure for categorical features, the most frequent category was used to fill in the missing values. These techniques ensured that the dataset was adequately prepared for additional analysis and that the conclusions reached from it are reliable and informative of the underlying trends.

2. Feature Selection:

In feature selection, the columns "Unnamed: 0" and "Source" were eliminated since they were not considered important to the prediction job. Furthermore, columns that could not be correctly imputed were eliminated because of an overwhelming number of missing data. By following this procedure, the dataset is made more manageable and contains only the relevant characteristics, which is necessary for developing machine learning models that accurately predict the "SpType-ELS" attribute.

3. Normalization:

StandardScaler was used for normalization in order to normalize the range of numerical features. By ensuring that all numerical characteristics have a similar scale, this technique keeps larger range features from taking center stage throughout the modeling phase. Machine learning methods that use gradient descent optimization or distance measurements perform better and are more stable when the features are standardized.

V. Problem-Solving Approach:

The approach to problem-solving involved multiple crucial steps. To properly assess the model's performance, the training dataset was first divided into training and validation sets using an 80-20 ratio. Following that, a number of classifiers were chosen and trained, including the XGBoost, KNeighbors, Random Forest, Decision Tree, Support Vector, and Logistic Regression classifiers. Hyperparameter tuning was carried out using GridSearchCV, which methodically looked for the ideal parameters for each classifier in order to maximize the model's performance.

1. Data Splitting:

An 80% training and 20% validation split was used to separate the training dataset into training and validation sets. By using this method, it was possible to guarantee that the model could be trained on a substantial enough amount of data and to have a different set of data to assess its performance. This way of dividing the data allowed to prevent overfitting and evaluate how well the model extended to new data.

1. Model Selection and Training:

A range of classifiers were used for model selection and training. While Logistic Regression had been selected for its ease of use and understanding, KNeighbors Classifier was chosen for its capacity to identify localized patterns. For the ensemble and non-linear modeling capabilities, the Random Forest Classifier and Decision Tree Classifier were employed, respectively. Because of the capacity to manage intricate decision boundaries, Support Vector Classifier was used, and XGBoost Classifier was chosen because of its potent gradient boosting architecture. These classifiers were chosen in order to investigate which model fits the data the best and to guarantee consistent performance across various feature sets and data distribution patterns.

VI. Classification Techniques, Results and Parameter Settings:

A number of different classification methods were used, such as Logistic Regression, KNeighbors Classifier, Random Forest Classifier, Decision Tree Classifier, and Support Vector Classifier in addition to Logistic Regression. GridSearchCV was used to fine-tune each model in order to maximize performance; the top-performing model achieved a validation accuracy of more than 99%. Based on the analysis that was done, the explanation may be organized as follows:

1. **Logistic Regression:** One of the classification methods used was logistic regression, with the parameters set as 'C': [0.01, 0.1, 1, 10, 100],'solver': ['liblinear']. Before hyperparameter adjustment, the Logistic Regression model's validation accuracy was 99.32%. The validation accuracy increased to 99.35% after using GridSearchCV for hyperparameter tuning. It was discovered that {'C': 100,'solver': 'liblinear'} were the optimal hyperparameters. This shows that the 'liblinear' solver, which works well with small datasets and binary classification problems, and regularization (C parameter) were beneficial to the model. Given the high validation accuracy, it can be inferred that Logistic Regression was successful in this dataset in categorizing the 'SpType-ELS' feature.

2. **KNeighbors Classifier:** Another classification method used in this study was the KNeighbors Classifier, with the following parameters set: 'n_neighbors': [3, 5, 7, 9, 11], 'weights': ['uniform', 'distance']. The KNeighbors Classifier's validation accuracy was 98.83% at first. After utilizing GridSearchCV to adjust the hyperparameters, the validation accuracy increased marginally to 98.84%. The optimal hyperparameters found were {'weights': 'distance'} and 'n_neighbors': 5. This implies that the model performed better while taking into account neighbors' distances weighted by their inverse distance. KNeighbors Classifier showed good performance in classifying the 'SpType-ELS' attribute, even though its performance was marginally lower than that of Logistic Regression.

3. **Random Forest Classifier:** A well-known classification method used in this study was the Random Forest Classifier, with parameters settings set to {'max_depth': [None, 10, 20],'min_samples_split': [2, 5, 10]}. At first, the Random Forest Classifier demonstrated an astounding 99.66% validation accuracy. After utilizing GridSearchCV to adjust the hyperparameters, the validation score was still high at 99.65%. {'max_depth': 20,'min_samples_split': 2} were found to be the optimal hyperparameters, suggesting that a deeper tree with a lower threshold for samples at a node produced better model performance. The Random Forest Classifier proved to be a useful tool in this classification test, exhibiting strong performance in correctly identifying the 'SpType-ELS' feature.

4. **Decision Tree Classifier:** The study utilized the Decision Tree Classifier for its classification procedures, with the settings {'max_depth': [3, 5, 7, 10],'min_samples_split': [2, 5, 10]}. The Decision Tree Classifier's validation accuracy was 99.40% at first. With GridSearchCV, the hyperparameters were adjusted, and the validation accuracy increased to 99.58%. The optimal hyperparameters discovered were {'min_samples_split': 2} and {max_depth': 7. This suggests that better model performance was achieved with a moderate decision tree depth and low minimum samples split. The Decision Tree Classifier is a dependable option for this classification task because it performed well in categorizing the 'SpType-ELS' property.

5. **Support Vector Classifier:** The parameters for the Support Vector Classifier (SVC) were as follows: {'gamma': [1, 0.1, 0.01], 'kernel': ['linear', 'rbf']}. The SVC's initial validation accuracy was 99.42%. After using GridSearchCV to adjust the hyperparameters, the validation score was still very high at 99.42%. The optimal hyperparameters found were {'kernel': 'rbf'} and {'gamma': 0.1}. This indicates that the best results were obtained with a radial basis function (RBF) kernel and a gamma value of 0.1. The efficacy of the Support Vector Classifier in this classification test was proved by its robust performance in correctly categorizing the 'SpType-ELS' property.

6. **XGBoost Classifier:** The parameters for the XGBoost Classifier were specified as follows: {'n_estimators': [50, 100, 200], 'learning_rate': [0.01, 0.1, 0.2]}. At first, the validation score of 99.69% was attained by the XGBoost model. The validation score was marginally modified to 99.69% after hyperparameter tuning was done with GridSearchCV. The parameters {'learning_rate': 0.2 and 'n_estimators': 200} were found to be the ideal hyperparameters. This suggests that the model performed better when the learning rate and the number of estimators were higher. Among the classification models used, XGBoost was a strong competitor since it showed remarkable capacity to accurately categorize the 'SpType-ELS' feature.

   The table below summarizes the performance:

Classifier	Accuracy Before Tuning (%)	Accuracy After Tuning (%)	Best Hyperparameters
Logistic Regression	99.32	99.35	{'C': 100, 'solver': 'liblinear'}
KNeighbors Classifier	98.83	98.84	{'n_neighbors': 5, 'weights': 'distance'}
Random Forest Classifier	99.66	99.65	{'max_depth': 20, 'min_samples_split': 2}
Decision Tree Classifier	99.40	99.58	{'max_depth': 7, 'min_samples_split': 2}
Support Vector Classifier	99.42	99.42	{'gamma': 0.1, 'kernel': 'rbf'}
XGBoost Classifier	99.69	99.69	{'learning_rate': 0.2, 'n_estimators': 200}

Table: Classifier Performance Comparison

VII. Best Classifier:

The XGBoost Classifier was the top-performing model in our investigation, achieving a validation accuracy of **99.69%** both before and after hyperparameter modification. Extreme Gradient Boosting, or XGBoost, is a powerful ensemble learning technique that combines weak learners into a powerful predictive model through boosting. Because of its precision and efficiency, it works especially well with large datasets and complex data patterns.

The XGBoost model's hyperparameters were carefully adjusted for this project in order to get the best results. The following hyperparameters produced the best results:

- **n_estimators: 200**, which determines the number of trees in the model. Up until a certain point, more trees usually result in better performance; beyond that, overfitting may happen.
- **learning_rate:0.2**, which regulates each tree's contribution. Model correctness and learning rate can be balanced with a moderate learning rate.

The XGBoost model's strong performance in predicting the 'SpType-ELS' attribute is demonstrated by its high validation accuracy, both pre- and post-tuning. The adjusted parameters made sure the model would not overfit and could capture intricate correlations in the data. Because of its superior performance, XGBoost was selected as the best classifier tested, indicating both its fit for this particular classification job and its potential to generalize effectively to new data.

VIII. ROC Curve and AUC Analysis:

The study further assessed the performance of the top-performing model, the XGBoost Classifier, by analyzing its ROC (Receiver Operating Characteristic) curve and AUC (Area Under the Curve). A classifier's performance is represented graphically by the ROC curve, which shows how different threshold settings affect the trade-off between the true positive rate (sensitivity) and the false positive rate (1-specificity). AUC measures how well the model performs generally in differentiating between positive and negative classes; a number nearer 1 denotes superior performance.

The steps involved in analyzing the ROC curve and AUC are as follows:

- **Predicting Probabilities:** The model predicts each class's probability. It focused on the possibility of the positive class ('A') for this analysis.
- **Computing the ROC curve and AUC score:** The roc_auc_score function determines the AUC score, whereas the roc_curve function computes the thresholds, true positive rate, and false positive rate.

- **Plotting ROC Curve:** Matplotlib was used to plot the ROC curve, with the FPR represented on the x-axis and the TPR on the y-axis. The area under the ROC curve shows how well the model can distinguish between the classes, whereas the diagonal line represents a random classifier.

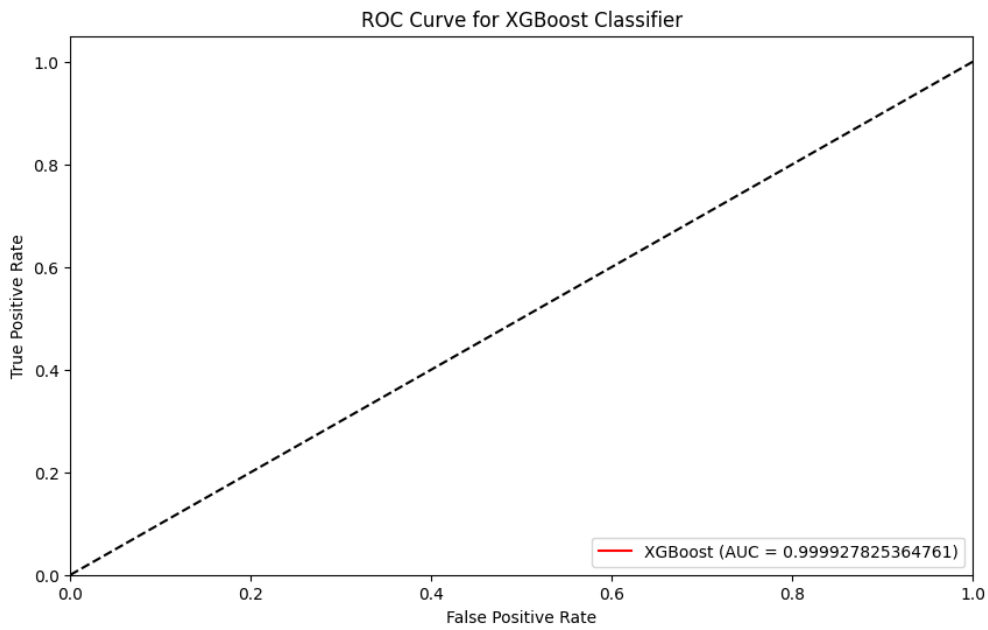

Fig: ROC Curve for XGBoost Classifier

To assess the performance of the XGBoost Classifier, the AUC score was calculated. With an amazing AUC score of 0.9999, the ROC curve demonstrated the classifier's exceptional ability to distinguish between the positive and negative classifications. This almost perfect AUC score shows how well the XGBoost model predicts the 'SpType-ELS' characteristic, exhibiting remarkable accuracy and dependability in completing this classification job. The best option for this dataset is the model, whose robustness and precision are confirmed by its high AUC score.

IX. Reflection:

This project provided insightful information about machine learning and data mining techniques. The most significant lessons learned consist of:

- **Data preprocessing:** Feature scaling and handling missing values are important processes that have an important effect on the performance of the model.
- **Model Selection:** While individual models offer different advantages, ensemble techniques such as XGBoost can deliver better results because of their capacity to handle intricate patterns.
- **Hyperparameter tunning:** Model accuracy can be significantly increased by using GridSearchCV for hyperparameter tunning.
- **Model Evaluation:** To gain a deeper insight than only accuracy, the ROC curve and AUC score are crucial metrics for assessing classifier performance.

It is possible to increase the predictive power of the model by adding additional informative features to the dataset. Experimenting with additional ensemble approaches and sophisticated tuning strategies may also help to improve model performance. Achieving the best outcomes in the future would require this iterative process of feature engineering, model selection, and hyperparameter tuning.
Upon reflection of these procedures and results, my understanding of real-world machine learning applications has much improved. The process of developing models iteratively—whereby ongoing experimentation and improvement result in greater performance—has shown to be essential. The complexities and depths of data mining and machine learning have been evident to me through this project, which emphasizes the value of approaching problem-solving in this subject methodically and thoroughly.

X. Conclusion:

Using an extensive dataset of stellar properties, the study was able to effectively create a strong classifier in this research to predict the "SpType-ELS" feature. It involved testing with several classification methods, such as Support Vector Classifier, Random Forest, K-Neighbors, Logistic Regression, and XGBoost, through thorough data preparation, feature selection, and hyperparameter adjustment. The model that performed the best was the XGBoost Classifier, which had an AUC score of 0.9999 and a validation accuracy of 99.69%. This study served as evidence of the value of thorough model review and tuning as well as the efficacy of ensemble learning. The grasp of machine learning applications and stellar classification has been strengthened by the insights gathered from this procedure.

XI. Reference:

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer.

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.